

Pixel-Level Protection System

End-to-end implementation reference: how uploaded images, PDFs, and multi-page documents are converted into pixel-authenticated, tamper-detectable digital identities — from upload through rasterization, cryptographic pixel binding, storage, and verification.

Scope: Image DNA pipeline, PDF/document rasterization & per-page protection, pixel HKCA (Phase 3A), dense per-pixel authentication (Phase 4E), local-DNA patch matching, forensic tamper localization, and verification/comparison engine.

Generated from the actual source implementation — every section cites the concrete file, service, and Prisma model involved.

Contents

1. System Overview	1
2. Upload & Detection	—
3. Image DNA Generation (15-Layer Pipeline)	—
4. Pixel-Level Cryptographic Protection (Spatial Auth / HKCA / Pixel1)	—
5. Local-DNA Patch & Feature Matching	—
6. Identity Embedding (Invisible Watermark)	—
7. PDF / Document Rasterization & Multi-Page Protection	—
8. Storage Model (Database & File Storage)	—
9. Verification & Comparison Engine	—
10. Tamper Localization & Region Detection	—
11. Crop / Resize Detection (Homography)	—
12. Similarity, Confidence & Tamper Scoring	—
13. End-to-End Flow Diagrams	—
14. File & Service Reference Index	—

1. System Overview

PINIT-DNA protects a file (image, PDF, or document) by generating a multi-layer cryptographic and perceptual fingerprint — its **DNA** — at upload time, then persisting that fingerprint so any later copy of the file can be compared against it to prove authenticity or localize exactly what changed.

For **images**, protection operates directly on pixels: cryptographic hashes of raw pixel bytes, perceptual hashes tolerant of re-compression, per-region keyed HMAC tags (HKCA), an optional dense per-pixel HMAC layer, and a patch-level feature index for crop/fragment recovery.

For **PDFs and documents**, the system treats the problem as "protect each page like a photo": every page is rasterized to a PNG image and pushed through the exact same pixel-protection pipeline a standalone image gets, then the protected pages are reassembled into a downloadable protected PDF.

Core design principle: the document-protection code does not duplicate the image pipeline — it calls it. A rasterized PDF page is, from the moment it is rendered, indistinguishable from any other uploaded PNG as far as the DNA engine is concerned. This is why the same tamper-detection and confidence-scoring logic applies uniformly to photos and to individual pages of a 50-page report.

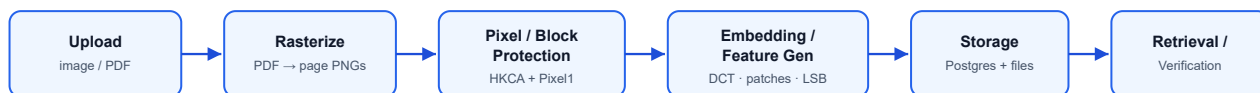


Figure 1 — High-level pipeline (images skip the "Rasterize" step)

2. Upload & Detection

Every upload enters through a single entry point, the `UniversalFileRouter` — `src/services/universal-file-router.ts`. It:

1. Runs `FileTypeDetector.detect()` (`src/services/file-type-detector.ts`) to classify the buffer by magic bytes / declared MIME type (IMAGE, PDF, DOCX, TXT, ...).
2. Routes `IMAGE` uploads to `DnaOrchestrator` directly.
3. Routes `PDF` / `DOCX` / other document types through `routeText()`, which creates a `PENDING` `DnaRecord` row, runs the type-specific text-DNA engine (e.g. `src/services/engines/pdf/pdf-dna-engine.ts`), then layers L7–L15 on top.

For a PDF specifically, after the text-DNA engine finishes, the router fires the page-level pixel-protection job (Section 7) as a background task — it does not block the HTTP response:

```
// src/services/universal-file-router.ts
if (fileType === 'PDF' && file.ownerUserId) {
  void documentPageProtectionService
    .protectPdfPages({ documentDnaRecordId: dnaRecordId, buffer: file.buffer, ... })
    .catch((err) => logger.warn('Document page protection skipped (non-fatal)', ...));
}
```

3. Image DNA Generation — the 15-Layer Pipeline

Every image (including every rasterized PDF page) is pushed through `DnaOrchestrator.generate()` — `src/services/dna.orchestrator.ts`. Layers 1–5 run in parallel; Layer 6 runs last because it embeds the record's own ID. Layers 7–10 and 11–15 run afterward.

Layer	Name / File	What it computes	Survives
L1	Cryptographic <code>layer1.cryptographic.ts</code>	<code>sha256Hash</code> of raw file bytes; <code>normalizedHash</code> = SHA-256 of the raw decoded RGB pixel bytes (via <code>sharp().removeAlpha().raw()</code>) — this is the literal "hash every pixel" fingerprint. Also a BLAKE3 hash.	Nothing — any pixel change produces a totally different hash. This is the exact-match layer.
L2	Structural <code>layer2.structural.ts</code>	Canny edge map, edge vectors, a 16-hex edge signature embedded into a "carrier image" via LSB for later structural diffing.	Minor recompression; breaks on crop/rotate/structural edits.
L3	Perceptual <code>layer3.perceptual.ts</code>	<code>pHash64</code> (32×32 greyscale → 2D DCT-II → top-left 8×8 AC coefficients → threshold at mean → 64 bits), <code>pHash256</code> (64×64 → 16×16 DCT block → 256 bits), <code>aHash64</code> (8×8 average threshold), <code>dHash64</code> (9×8 adjacent-pixel comparison).	JPEG recompression, resize, brightness/contrast shifts, PNG↔JPEG conversion.
L4	Semantic <code>layer4.semantic.ts</code>	RGB + HSV colour histograms, dominant colours, a 12-char colour fingerprint.	Small edits; breaks on colour grading / filters / palette swap.
L5	Metadata <code>layer5.metadata.ts</code>	EXIF/IPTC/XMP, device make/model, capture time, GPS, a metadata hash.	Nothing structural — this is provenance evidence, not content proof.
L6	Steganography <code>layer6.steganography.ts</code> , <code>ownership-watermark.ts</code>	Tiles a compact ownership signature (DNA id, vault id, owner id, upload metadata) across every blue-channel LSB (so ~20% crops can still recover identity), plus <code>contentSealHmac</code> — an HMAC over the L1–L5 digests ("EDS identity seal").	Crops down to ~20% of the image; broken by any lossy recompression of the blue channel.
L7–L10	Behavioral / Relationship / Origin / Evolution <code>layer7.behavioral.ts</code> ... <code>layer10.evolution.ts</code>	Upload session/device fingerprint, links to related DNA records, origin bundle, Merkle-chained mutation/version log.	Session-specific — weight 0 in content-similarity scoring (see Section 9).
L11–L15	Deepfake / DCT Watermark / Custody / ZK Proof / Biometric <code>layers-11-15.service.ts</code>	AI-generation heuristics, a second invisible DCT-frequency-domain watermark, legal chain-of-custody record, zero-knowledge ownership commitment, optional biometric bind.	Enrichment / evidentiary layers, not used for content-match scoring.

Each layer persists to its own Prisma table (`crypto_layers`, `structural_layers`, `perceptual_layers`, ...) keyed 1:1 by `dnaRecordId` — see Section 8 for the full schema.

4. Pixel-Level Cryptographic Protection

Layers 1–15 fingerprint the image as a whole. A separate subsystem — **Spatial Auth** (`src/services/spatial/`) — authenticates the image *region by region*, so a probe copy can be compared cell-by-cell and any tampered area pinpointed. It has three tiers of increasing granularity, all keyed by a per-image HKDF-derived secret so tags cannot be forged without the server's master secret.

4.1 Phase 1 — Block Auth (coarse, ~64–128px blocks)

The base tier splits the image into large blocks (primary scale 64px, with 128px also enrolled) and produces block-level leaves that are Merkle-rooted into `spatial_auth_packages.blockBlob`. This gives cheap, fast coarse tamper detection before the finer tiers are checked.

4.2 Phase 3A — HKCA: Hierarchical Keyed Cell Authentication (8×8 cells)

File: `src/services/spatial/pixel-auth/`. This is the primary pixel-region layer:

1. **Grid** — `buildPixelCellGrid()` (`cell-grid.ts`) divides the image into an 8×8-pixel cell grid (edge cells hold their true, possibly smaller, width/height).
2. **Per-image key** — `deriveSpatialPixelKey()` (`key-derivation.ts`) runs HKDF-SHA256 over the server master secret, salted with `dnaRecordId` and info bytes built from `ownerUserId`, `globalDnaRef`, and a `keyId`. No key material is ever stored — it is always re-derived.
3. **Per-cell HMAC tag** — `computePixelCellAuthTag()` (`cell-auth.ts`) hashes the raw RGB bytes of that one cell (SHA-256) then HMAC-SHA256s a canonical message — `{algoVersion, dnaRecordId, globalDnaRef, cellSize, cellId, x, y, w, h, contentHash}` — with the derived key, truncated to `tagBytes` (8 or 16, default 8).
4. **Compact blob** — `encodePixelAuthBlob()` (`pixel-blob.ts`) packs every cell leaf into one binary blob (magic `SPX1`), sorted and validated for canonical ascending `cellId` order — a few hundred KB per image, not one DB row per pixel.
5. **Merkle root + root MAC** — `pixelMerkleRootHex()` builds a binary Merkle tree over every cell leaf's hash; `computePixelRootMac()` then HMACs the root together with image dimensions, owner, and `globalDnaRef` under a *second* HKDF-derived key (`pixel-merkle.ts`). This root MAC is what actually gets compared at verify time — a single 32-byte check proves the entire blob is intact before any per-cell work happens.

Why HMAC instead of embedding bits in pixels: HKCA is deliberately *server-side only* — no pixel values are altered to store the tag (the code comments call this "no GLPM / no pixel embedding"). The tags live in `spatial_auth_packages`, keyed to the image's own content hash, so the original file's pixels are never touched by this layer.

Verification & fine localization

`verify.ts` — `verifyPixelAuthLayer()` :

1. Recompute and check the `pixelRootMac` (fails fast if the stored package was tampered).
2. Decode the blob, recompute the Merkle root, and confirm it matches `pixelAuthRoot`.
3. Re-derive the per-image key and recompute the HMAC tag for *every* 8×8 cell of the candidate image.
4. Compare each candidate tag to the stored tag with `crypto.timingSafeEqual` — any mismatch flags that cell `TAMPERED`.

5. `fine-map.ts` then runs 4-connected-component labelling over the failed cells (`findConnectedFineRegions()`) to group them into bounding-box regions, and computes `tamperedAreaPercentage` .

4.3 Phase 4E — Pixel1: Dense Per-Pixel HMAC (every single pixel)

The most granular — and most expensive — tier: `buildPixel1AuthPackageFromRgb()` generates one keyed HMAC tag *per pixel* (not per 8×8 cell), stored in `spatial_auth_packages.pixel1AuthBlob` . For a 1240×1755 page that is ~2.18M individually-tagged pixels.

Cost & where it is skipped: enrolling Pixel1 for one image measured ~17–40 seconds and writes a ~17MB blob.

`DnaOrchestrator.generate()` accepts a `skipSpatialPixel1` flag in its `universalCtx` (`src/services/dna.orchestrator.ts`), and document-page protection (Section 7) always passes it — a multi-page PDF would otherwise pay this cost once per page, per enrollment pass. Standalone image uploads still get full Pixel1 coverage.

5. Local-DNA Patch & Feature Matching

HKCA proves "this exact image, unmodified." **Local-DNA** (

`src/services/forensics/local-dna-patch-generator.service.ts`) answers a different question — "does a crop, screenshot, or small fragment of this image appear inside another image?" — by fingerprinting the image at patch level.

Multi-scale patch grid

`generateMultiScaleGrid()` tiles the image at four fixed scales (16/32/64/128px, configurable via `src/config/local-dna.ts`), plus overlapping pyramid tiles (64→512px, 50% stride) so a 20–30% crop can still be recovered. Each patch gets:

- `pHash16`, `dHash8`, `aHash8` — patch-local perceptual hashes.
- `edgeSignature` — Canny-style edge fingerprint.
- `colorVector` — mean RGB.
- `frequencySig`, `textureSig` — frequency/texture descriptors.

Matching (`patchFingerprintsMatch` / `patchDenseMatch` in the same file) first checks Hamming distance on `pHash16` (threshold from `PINIT_LOCAL_DNA_HAMMING_THRESHOLD`, default 14), then, for borderline cases, requires 3-of-6 secondary signals (`dHash`, `aHash`, `edge`, `colour` distance < 45, frequency, texture) to agree — a voting scheme rather than a single brittle threshold.

Fragment-splice detection

`src/services/forensics/fragment-splice-detector.service.ts` answers "is a small piece of a protected original pasted into an unrelated image?" It clusters matched patches into spatially-coherent islands (pixel-proximity BFS), then gates each island on: minimum patch count, the island's probe bounding-box staying small relative to the whole probe canvas (a large matched region is a crop, not a splice — left to whole-image search instead), and internal spatial offset consistency.

ORB features & crop-resistant tile search (Python side)

`python-ai/services/forensic_scanner/service.py` extracts ORB keypoints/descriptors (`cv2.ORB_create`) and indexes overlapping tiles into a FAISS index (`search_tiles()`) so a crop-resistant candidate search can run without brute-forcing every stored image — this is what `forensicScannerService.indexVaultTiles()` populates from Node (`src/services/forensics/local-dna-index.service.ts`).

6. Identity Embedding — Invisible Watermark

Everything above is *read-only* fingerprinting — it never changes the file's pixels. A separate step, run at **vault-store time** (`src/services/identity/identity-embedding.service.ts`), orchestrated by `identity-embedding-pipeline.service.ts`), actually writes a recoverable identity into the file:

Signature format

```
PINIT-DNA:v1:{dnaId}:{vaultId}:{ownerUserId}:{hmacHex}
hmac = HMAC-SHA256(dnaId + ":" + vaultId + ":" + ownerUserId, LSB_SIGNATURE_SECRET)
```

Per-type embedding method

File type	Method
PNG / JPG / WEBP	<code>embedImageLSB()</code> — payload (<code>"PDNA"</code> magic + 4-byte length + signature bytes) is written one bit per pixel into the blue-channel LSB , left-to-right / top-to-bottom, plus an appended binary tail marker as a fallback extraction path.
PDF	PDF <code>Keywords</code> / <code>Subject</code> metadata property + the same binary tail marker appended after the PDF trailer.
DOCX / XLSX / PPTX	A custom XML property injected into the OOXML ZIP container (<code>docProps/custom.xml</code>).

The full vault pipeline (`identity-embedding-pipeline.service.ts`) layers several protections on top of the raw signature: an identity token, a recovery token, an invisible watermark (`vaultWatermarkEngine`), the cryptographic signature above, and a signed integrity manifest appended as a tail block — each verified with `extractAndVerify()` before the pipeline reports success.

7. PDF / Document Rasterization & Multi-Page Protection

This is the layer that makes "protect every page like a photo" real. Two services work together:

7.1 Rasterization

`python-ai/services/document_rasterizer/service.py` uses PyMuPDF (`pymupdf`) to render each PDF page to a PNG at a configurable DPI (default 150, capped at a configurable max page count — see `src/config/document-pixel-protection.ts`):

```
matrix = fitz.Matrix(dpi/72, dpi/72)
doc = fitz.open(stream=buffer, filetype="pdf")
for i in range(min(doc.page_count, max_pages)):
    pix = doc.load_page(i).get_pixmap(matrix=matrix, alpha=False)
    pages.append({ pageNumber: i+1, width: pix.width, height: pix.height,
                  imageBase64: base64(pix.tobytes("png")) })
```

Exposed as `POST /document/rasterize` in `python-ai/main.py`, called from Node via `aiService.rasterizeDocument()` (`src/services/ai/ai-embeddings.service.ts`).

7.2 Per-page protection — two phases

`src/services/documents/document-page-protection.service.ts` runs the actual per-image pipeline (Sections 3–4) once per rendered page, in two distinct phases:

Phase A — DNA-generate time (fast, fire-and-forget): `protectPdfPages()`

For each page: run `DnaOrchestrator.generate()` on the raw page PNG (full 15-layer DNA, HKCA enrolled on raw bytes, Pixel1 skipped), then link the new page's own `DnaRecord` back to the parent document via two self-relation fields added to `dna_records`:

```
documentDnaRecordId  String?  // parent document's DnaRecord.id
pageNumber           Int?
```

Then `localDnaIndexService.buildIndex()` builds the patch grid (Section 5) for that page. This phase is idempotent — it checks for an existing (`documentDnaRecordId`, `pageNumber`) row before creating a new one, so re-triggering the same document never duplicates page records.

Phase B — vault-store time (background): `protectAndAssembleForVault()`

Called from `src/services/vault/vault.service.ts` after the raw PDF is already safely encrypted and vaulted (so this never blocks the upload response — see the callout below). For each page it:

1. Reuses the Phase-A page `DnaRecord` if present, else creates it.
2. Embeds a per-page identity signature into the page's raw pixels via `identityEmbeddingService.embed()` (Section 6) — `dnaId` = that page's own record, `vaultId` = the parent document's record, so a leaked page can be traced back to its source document.
3. Re-enrolls HKCA and rebuilds the local-DNA patch index on the *post-embed* bytes (embedding changes pixel values, so the stored tags must match what actually ships) — mirroring exactly how standalone image vault-store re-enrolls on post-embed bytes.

The protected pages are then reassembled into one PDF with `pdf-lib` (`PDFDocument.create()`), one page per protected PNG, sized in points at `pixelWidth × 72 / dpi`, re-encrypted under the *same* `vaultId`, and swapped in for the raw file already in storage — updating `vault_records.encryptedFilePath` / `ivHex` / `authTagHex` in place. The existing protected-download endpoint then transparently serves the pixel-protected pages with no separate download path.

Why Phase B is background, not synchronous: the full per-page pipeline (DNA + HKCA + patch index) took ~55–110 seconds per page in measurement. A 5-page document would exceed the frontend's request timeout if `vault.service.ts` awaited it inline — this was fixed by making the PDF upgrade fire-and-forget (`upgradePdfInBackground()`) after the fast raw-file store returns.

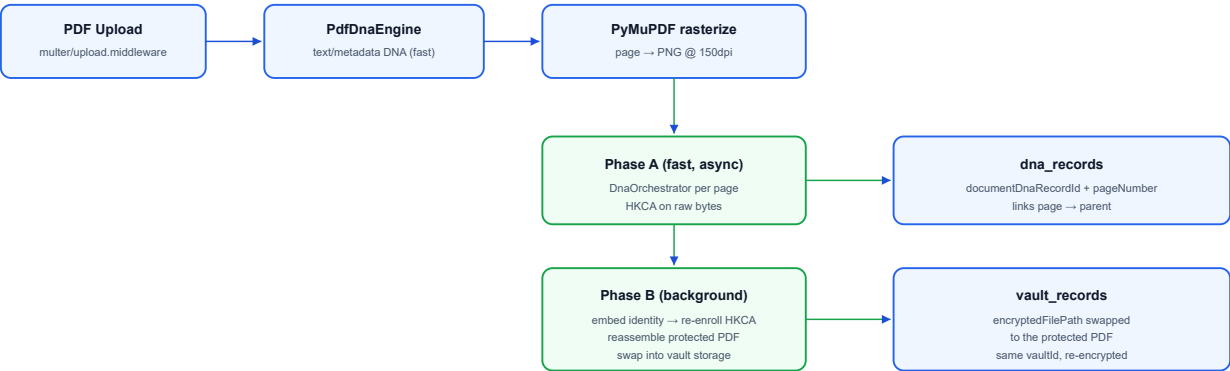


Figure 4 — PDF multi-page protection: text-DNA is fast and synchronous; per-page pixel protection runs in two background phases so the upload/store requests stay fast.

8. Storage Model

All protection data lives in PostgreSQL via Prisma (`prisma/schema.prisma`). Encrypted originals live in Supabase Storage (production) or local disk under `./vault/encrypted` (dev) — never inside the database.

Table	Key fields	Purpose
<code>dna_records</code>	<code>id</code> , <code>sha256Hash</code> , <code>status</code> , <code>fileType</code> , <code>ownerUserId</code> , <code>universalFingerprints</code> (JSON), <code>documentDnaRecordId</code> , <code>pageNumber</code>	One row per protected image <i>or</i> per document page. Self-referential for document pages.
<code>crypto_layers</code>	<code>sha256Hash</code> , <code>normalizedHash</code> , <code>blake3Hash</code>	L1
<code>structural_layers</code>	<code>edgeMapB64</code> , <code>edgeSignature64</code>	L2
<code>perceptual_layers</code>	<code>pHash64</code> , <code>pHash256</code> , <code>aHash64</code> , <code>dHash64</code>	L3
<code>semantic_layers</code>	<code>histograms</code> , <code>colorFingerprint</code>	L4
<code>metadata_layers</code>	EXIF/IPTC/XMP , <code>metadataHash</code>	L5
<code>stego_layers</code>	<code>payloadHmac</code> , <code>ownershipSignature</code> (JSON), <code>ownershipTileCount</code>	L6
<code>spatial_auth_packages</code>	<code>blockBlob</code> (Phase 1), <code>pixelAuthBlob</code> / <code>pixelAuthRoot</code> / <code>pixelRootMac</code> (Phase 3A HKCA), <code>pixel11AuthBlob</code> / <code>pixel11AuthRoot</code> (Phase 4E), <code>globalDnaRef</code> , <code>keyId</code>	One compact row per protected image/page — <i>not</i> one row per pixel.
<code>local_feature_indexes</code> + <code>local_dna_patches</code>	<code>patchSize</code> , <code>gridCols/Rows</code> , <code>globalPHash</code> , <code>orbDescriptors</code> ; per-patch <code>pHash16</code> , <code>dHash8</code> , <code>aHash8</code> , <code>edgeSignature</code> , <code>colorVector</code>	Section 5 patch grid, indexed for crop/fragment search.
<code>vault_records</code>	<code>encryptedFilePath</code> , <code>encryptionAlgorithm</code> (AES-256-GCM), <code>ivHex</code> , <code>authTagHex</code> , <code>originalSizeBytes</code>	The encrypted file itself (or its storage pointer) plus decrypt parameters.
<code>document_lineage</code>	<code>fromDnaRecordId</code> , <code>toDnaRecordId</code> , <code>relation</code> , <code>confidence</code>	Recorded DUPLICATE/SIMILAR/CROP_DERIVATIVE/... relationships discovered during comparisons.

Retrieval for verification

Given a `vaultDnaRecordId` , `src/services/forensics/investigation-stored-dna-source.service.ts` resolves the authoritative stored fingerprint — preferring the sealed `enterpriseDnaPackage` (immutable JSON snapshot of all layer digests, Section 9) over re-fingerprinting live vault bytes, so verification is fast and does not require decrypting the original file for every comparison.

9. Verification & the Comparison Engine

`src/services/verification/comparison-engine.ts` — `ComparisonEngine.compare(fpA, fpB)` — is the core layer-by-layer matcher. Both a stored fingerprint (A) and a probe fingerprint (B) carry the same 15-layer shape; the engine compares layer-for-layer and produces one verdict.

Per-layer thresholds & weights

Layer	Weight	Match threshold
L1 Cryptographic	0.35	1.00 (exact only)
L2 Structural	0.20	0.80
L3 Perceptual	0.20	0.75
L4 Semantic	0.10	0.70
L5 Metadata	0.05	0.60
L6 Signature	0.10	0.90
L7–L10	0 (shown, not scored)	0.50 / 1.00

`overallScore =  $\sum$  (similarityScore  $\times$  weight)` across non-skipped layers. Classification:

```
if (L1 matched exactly && overallScore >= 95) → DNA_MATCH
else if (overallScore >= 55) → SIMILAR
else → DIFFERENT
```

Tampering pattern detection

`analyzeTampering()` inspects which layers changed and classifies the *kind* of change with severity, not just a binary flag:

- **Re-encode** (MEDIUM): L1 bytes differ, L3 perceptual similarity $\geq 0.92 \rightarrow$ same content, different encoding.
- **Minor content edit** (LOW): L1 differs, L3 similarity 0.70–0.91.
- **Minor structural edit** (LOW): L1 & L2 differ but L3 stays ≥ 0.90 .
- **Metadata manipulation only** (MEDIUM): L5 differs, L1–L4 all match — content unchanged, provenance edited.
- **Signature forgery attempt** (CRITICAL): L1–L5 all match but L6 differs — content is untouched but the embedded identity seal doesn't verify.
- **Heavy structural modification** (HIGH): L2 *and* L3 similarity both < 0.50 .
- **Semantic/colour-only edit** (LOW): L4 differs while L2 and L3 stay ≥ 0.80 — filter/colour-grade.

`tamperingDetected = indicators.length > 0 || L6 changed`.

Document-page verification nuance: L5 (metadata) and L6 (the per-record embedded signature) are inherently unable to match between a freshly re-rasterized probe page and a stored fingerprint, even for byte-identical content — L6 in particular is deliberately unique per `dnaRecordId`. Document tamper-checking (`verifyProtectedDocument()` in `document-page-protection.service.ts`) therefore reads L1 and L3 directly from `layerComparisons` rather than trusting the generic `classification` / `tamperingDetected` fields: a page counts as unchanged if L1 matched exactly, or L3 similarity ≥ 0.92 (same "re-encoded, content identical" bar the engine itself uses).

Enterprise comparison (V2)

`src/services/verification/enterprise-comparison-engine.service.ts` runs an additional 15-layer domain-scored comparison (`identity` / `evidence` / `ownership` / `trust` domains) against the sealed `enterpriseDnaPackage` when available, feeding `identityScore` / `evidenceScore` /etc. into the final report alongside the legacy engine's score — both run so the legacy `overallConfidenceScore` (which downstream Acceptance logic depends on) stays stable while the richer domain breakdown is available for investigators.

10. Tamper Localization & Region Detection

Two independent localization mechanisms exist, used in different situations.

10.1 Cryptographic localization (no reference bytes needed)

HKCA (Section 4.2) re-derives the per-image key and re-computes the HMAC of every 8×8 cell of the *candidate* image, compares each to the stored tag, and feeds the pass/fail map into `findConnectedFineRegions()` (`fine-map.ts`) — a 4-connected flood fill over failed cells that merges them into bounding-box regions with a cell count and stats (`tamperedAreaPercentage` , `exactPassRate`). This works even when only the protected DNA record is available — no separate reference file needs to be fetched or decrypted.

10.2 Pixel-diff localization (reference image available)

`python-ai/services/forensic_scanner/service.py` — `localize_tamper(probe, reference)` : resizes the reference to the probe's dimensions, computes `cv2.absdiff()` , converts to greyscale, thresholds at 28, applies a 5×5 morphological close to merge nearby noise, then `cv2.findContours()` to extract up to 8 bounding-box regions ($\geq 200\text{px}^2$ each). Produces `modifiedPercent` , a JET-colormap heat overlay (base64 PNG for the investigator UI), and per-region classification via `_classify_region_change()` .

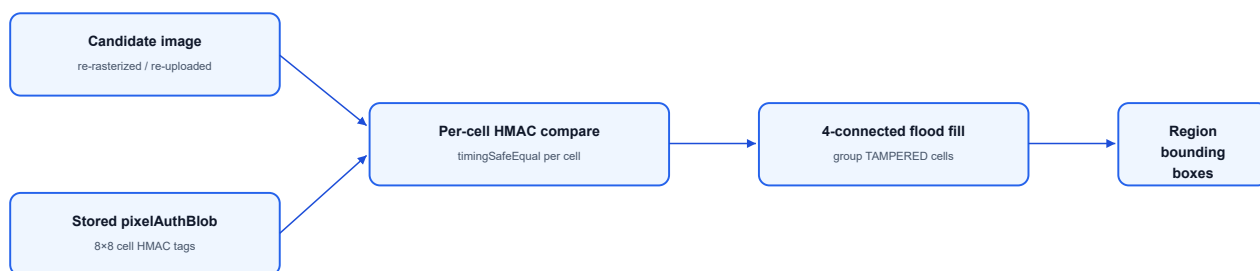


Figure 3 — Pixel/region-level tamper localization (HKCA path — no reference file required)

11. Crop & Resize Detection (Homography)

`python-ai/services/forensic_scanner/service.py` — `detect_crop_homography(probe, reference)` :

1. Decode both images to grayscale (max dimension 1280px) and normalize.
2. Extract up to 2500 ORB keypoints/descriptors from each (`cv2.ORB_create`).
3. Match with a brute-force Hamming matcher, cross-check enabled (`cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)`), keep the best 80 matches under distance 50.
4. If ≥ 8 good matches: fit a homography with `cv2.findHomography(src, dst, cv2.RANSAC, 5.0)` — RANSAC separates true geometric-transform inliers from spurious matches.
5. `sharedRegionPercent` is derived from the inlier ratio (inliers / good matches) blended with inlier coverage of the probe's own keypoints; `cropPercent = 100 - sharedRegionPercent` .

Below 8 good matches, the function falls back to a coarse keypoint-count ratio rather than attempting an unstable homography fit.

This is the mechanism behind "X% cropped / Y% still visible" reporting in investigation results, and is what identifies that two images are the *same underlying photo* even after a crop or perspective change that would break a simple pixel hash comparison.

12. Similarity, Confidence & Tamper Scoring — Summary

Signal	Where computed	What it measures
Hamming similarity (pHash/dHash/aHash)	Layer 3, local-DNA patches	Perceptual closeness, tolerant of re-encoding/resize
HKCA cell pass rate	Spatial Auth 3A	Fraction of 8×8 regions cryptographically intact
Merkle root / root MAC match	Spatial Auth 3A	Binary — was the whole HKCA package tampered
Weighted overall score (0–100)	ComparisonEngine.compare()	Blended L1–L6 similarity → DNA_MATCH / SIMILAR / DIFFERENT
ORB inlier ratio → sharedRegionPercent	forensic_scanner (Python)	Geometric overlap after crop/rotate/perspective change
modifiedPercent (pixel diff)	forensic_scanner.localize_tamper	Literal % of probe pixels differing from a reference
Fusion score	<code>_fusion_weights()</code> , forensic_scanner	pHash 0.14, tile 0.18, ORB 0.14, CLIP 0.22, texture 0.08, watermark 0.12, dna 0.12 — blended multi-signal match confidence

Distinguishing original vs. modified is never a single check — it is the combination above: L1 exact-match settles "byte-identical or not"; L3/pHash settles "same visual content despite re-encoding"; HKCA settles "exactly which region changed, cryptographically, with no reference file"; ORB/homography settles "same photo, cropped/rotated/perspective-shifted"; and the tampering-pattern rules (Section 9) turn the raw layer diffs into a human-readable verdict (re-encode vs. minor edit vs. forged signature vs. heavy structural change).

13. End-to-End Flow Diagrams

13.1 Protection / Enrollment Flow

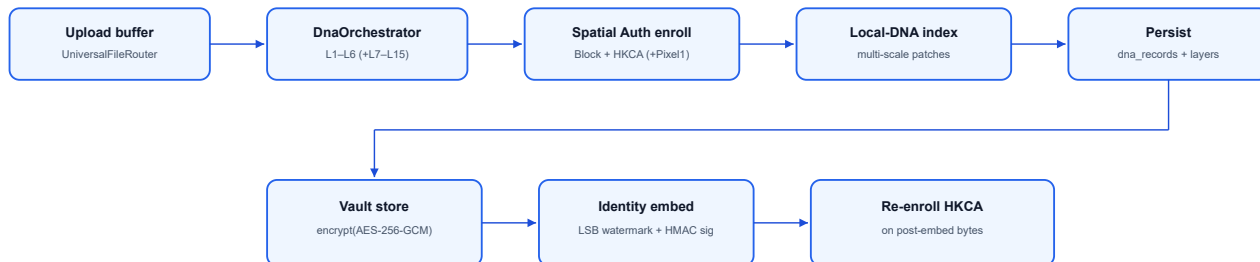


Figure A — Enrollment: fingerprint first, encrypt, embed identity, then re-enroll HKCA on the bytes that actually leave the vault.

13.2 Verification / Matching Flow

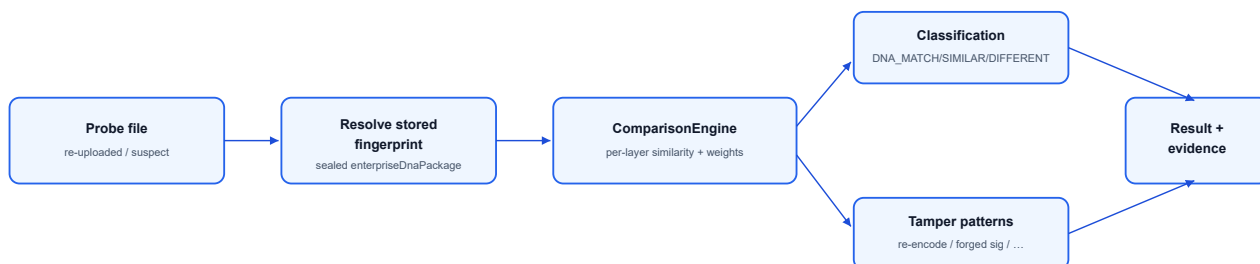


Figure B — Verification: resolve the SSoT stored fingerprint (no decrypt needed for scoring), compare per layer, derive both a classification and a human-readable tamper pattern.

14. File & Service Reference Index

Concept	File
Upload entry point / type detection	<code>src/services/universal-file-router.ts</code> , <code>src/services/file-type-detector.ts</code>
Image 15-layer DNA orchestrator	<code>src/services/dna.orchestrator.ts</code>
Layers 1–6	<code>src/services/layers/layer1.cryptographic.ts</code> ... <code>layer6.steganography.ts</code>
Layers 7–10 / 11–15	<code>src/services/layers/layer7.behavioral.ts</code> ... <code>layer10.evolution.ts</code> , <code>layers-11-15.service.ts</code>
Spatial Auth — enroll / persist	<code>src/services/spatial/enroll.service.ts</code>
HKCA (Phase 3A) internals	<code>src/services/spatial/pixel-auth/{cell-grid,cell-auth,pixel-blob,pixel-merkle,key-derivation,fine-map,verify,package-builder}.ts</code>
Dense per-pixel (Phase 4E)	<code>src/services/spatial/pixel1-*.ts</code> (via <code>enroll.service.ts</code>)
Local-DNA patch generation / index	<code>src/services/forensics/local-dna-patch-generator.service.ts</code> , <code>local-dna-index.service.ts</code>
Fragment/splice detection	<code>src/services/forensics/fragment-splice-detector.service.ts</code>
Identity embedding (LSB/PDF/OOXML)	<code>src/services/identity/identity-embedding.service.ts</code> , <code>identity-embedding-pipeline.service.ts</code>
Vault encrypt/store/retrieve	<code>src/services/vault/vault.service.ts</code>
Document rasterization (Python)	<code>python-ai/services/document_rasterizer/service.py</code>
Document page protection (2-phase)	<code>src/services/documents/document-page-protection.service.ts</code>
PDF/DOCX text-DNA engines	<code>src/services/engines/pdf/pdf-dna-engine.ts</code> , <code>engines/docx/docx-dna-engine.ts</code>
Comparison / classification engine	<code>src/services/verification/comparison-engine.ts</code> , <code>dna-comparison.service.ts</code>
Enterprise (V2) comparison	<code>src/services/verification/enterprise-comparison-engine.service.ts</code>
Leaked-file / re-upload detection	<code>src/services/forensics/leaked-file-verify.service.ts</code>
Forensic scanner — ORB/homography/pixel-diff (Python)	<code>python-ai/services/forensic_scanner/service.py</code>
Prisma schema (all models above)	<code>prisma/schema.prisma</code>

PINIT-DNA — Pixel-Level Protection Technical Documentation. Generated directly from the source implementation for engineering reference. Sections and diagrams reflect the code as of the current working tree; enable `PRISMA_LOG_QUERIES` / consult the referenced files directly for line-level detail beyond what is summarized here.